

Introduction

Q1. Explain what is meant by software engineering? What are the four major phases in software engineering?

Ans) It is the discipline of building, designing, testing, deploying and maintaining software systems in the most systematic, efficient way. The systematic approach helps in complexity handling and improves productivity and quality of product.

The four major phases of software engineering are:

- Requirements Engineering
- Architecture design
- Implementation
- Quality Assurance

Q2. In what ways is the Waterfall model different from Spiral model

Ans)

-The spiral model focuses on iterative development whereas in the waterfall model, there is no iteration between phases.

- In the spiral model, User feedback is given at the end of every loop, which the team can work on in the next loop. In the waterfall model, the user is shown the product only at the end.

-Spiral model is flexible when it comes to changing requirements and can easily respond and adapt to changes while waterfall model cannot adapt to changing requirements.

- With the spiral model, it becomes easy to validate requirements due to constant feedback.

Requirements could be misunderstood in waterfall models leading to the system failing.

- Waterfall model is easy to manage while Spiral model is complex to manage.

Q3. What are the 3 main roles in Scrum and explain what is meant by a sprint and Sprint review and Scrum Retrospect?

Ans)

The three main roles in Scrum are:

- Product owner: The person who decides what product to build and decides what is important for the project.
- Scrum Master: The scrum master ensures that the team follows the rules of the scrum.
- Development Team: A team that builds the product in increments.

A sprint is a fixed duration iteration in Scrum, usually 2-4 weeks, where the development team works on a small piece of the project in order to create a product increment. The goal of a sprint is to deliver working software that adds value.

In the sprint review, the product increment is demonstrated to the stakeholders and users. The stakeholders inspect the increment, feedback is collected and Product Backlog is updated.

Q4) "Scrum follows the principles of Agile." Elucidate.

Ans) Scrum follows the principles of Agile in the following way:

- Scrum emphasized collaboration and team communication over following a strict procedure.
- Product Owners work closely with stakeholders to set prioritize work
- Scrum focuses on providing working software increments every sprint/iteration over piles of documents.
- Scrum is very flexible when it comes to changing requirements since the changes can be updated in Product Backlog and be implemented in the next sprint.

Requirements Engineering

1) What does a Requirement engineer do? Give a detailed answer. (4 marks)

- Gives requirements for a certain system which is needed or wanted by a stakeholder.
- Documents the requirements for remembrance and reusage of the data.
- Validates the requirements, by which the progress of certain project is updated.
- Manages all the requirements so that the requirements are persistent and not forgotten.

2) What is the importance of documenting requirements (7 - 8 marks).

- **Persistence**
During a project, a **large amount of different types of information** are elicited and developed. Without proper documentation (part of) this information would be forgotten. Moreover, the stakeholders involved can hardly recall and reflect on all this information.
- **Common reference**
Documentation of the relevant information establishes a common reference for all stakeholders.
- **Promotes communication**
The common reference supports communication among the stakeholders and promotes discussions.
- **Promotes objectivity**
Unlike the verbal exchange of information, the exchange of documented information is generally less amenable to unwanted alteration of the information, e.g. due to subjective interpretation and subjective bias in the communication of the information.
- **Support training of new employees**
If a new stakeholder joins the project, the documented information provides an excellent basis for getting familiar with the project. For example, the stakeholder can selectively access the relevant information needed for performing a specific task or can get an overall overview of the system.
- **Preserve expert knowledge**
Typically, not all project members know all relevant aspects of a system such as all relevant technologies or all relevant context aspects. Documenting such expert knowledge makes this knowledge available to all project participants and hence reduces the dependence on individual experts.
- **Supports problem reflection**
When documenting information, the author is forced to structure the information in an appropriate way. Hence the author has to reflect on the information which often leads to the identification of gaps and inconsistencies

3) What are the types of Representation? Include their pros and cons (5 marks).

Textual such as language text, structured text and templates. These formats bring uniformity but sometimes they fill up space with unwanted information and use different formats which is confusing. They also do not help in visualizing a certain system.

Model based such as, structure perspective, function perspective and behavior perspective. With this format we can visualize a system and helps in communication but lacks explanation and not easily understood at first glance

Combined, text and models which solve the issues from both the ends

- 4) Give an example about Lastenheft and Pflichtenheft (6 marks)

Abstract Requirement in the Lastenheft:

R-L-12: The safety system shall offer user and access management.

Detailed Requirements in the Pflichtenheft:

R-8: The safety system shall differentiate between the administrator and user of the system.

...

R-14: The administrator shall be able to create new user accounts.

R-15: The system shall support the organization of user accounts in user groups.

R-16: The administrator shall be able to assign and revoke access authorizations.

...

R-23: The user shall be able to change his/her personal access code.

...

- 5) What is the quality criteria for specifications? (8 marks)

Affordable: There exists a feasible solution that satisfies the complete set of requirements within the lifecycle constraints (e.g. cost, time, legal). based on [ISO/IEC/IEEE 29148]

Bounded: The set of requirements is defined within the system scope, i.e. the requirements specification contains no "gold plating". based on [ISO/IEC/IEEE 29148]

Clearly structured (comprehensible): The requirements specification supports selective reading and should thus be clearly structured and comprehensible. based on [Pohl and Rupp 2011]

Complete: The requirements specification contains all relevant requirements, and each requirement is completely specified. No TBDs etc. are included. based on [ISO/IEC/IEEE 29148, IEEE 830-1998]

Consistent: All single requirements are internally consistently defined. There are no contradictions between the requirements. based on [ISO/IEC/IEEE 29148]

Modifiable and extendable: The requirements specification is easy to modify and extend. The document structure and style should support simple, complete and consistent modification.

based on [Pohl and Rupp 2011, IEEE 830-1998]

Traceable: The relationships between different versions of requirements specifications and other documents (e.g. business process models, architecture, test cases, user manuals, standards) should be traceable.

based on [Pohl and Rupp 2011]

Unambiguous: Each single requirement on its own, as well as the entire requirements specification allow only one valid interpretation. based on [Pohl and Rupp 2011]

6) Who are relevant stakeholders? (2 marks)

Sponsorers, Developer, Clients, User and Authorities

Conceptual Modelling

Question 1:

Explain what is meant by a conceptual model and how it differs from a technical or implementation-oriented description of a system.

Answer 1:

A conceptual model is an abstract representation of a part of reality (universe of discourse) for representing a specific purpose, that focuses on relevant concepts and relationships for understanding and communication, independent of technical realization. It is created to reduce complexity.

In contrast, an implementation model is a description that includes technical or implementation details which focuses on how the system is realized in practice. It includes concrete design decisions, technical details, and constraints related to programming languages, platforms, or architectures. While a conceptual model explains what the system is about, an implementation model explains how it is built.

Question 2:

During a team meeting, a developer quickly sketches boxes and lines on a whiteboard to explain how a complex system works, without using any formal notation. Can this sketch be considered a conceptual model? Justify your answer.

Answer 2:

Yes, this whiteboard sketch can be considered a conceptual model. Even though it does not use any formal notation, it is an intentional abstraction of the system that is created for a specific purpose here, to explain how the system works to other team members.

Question 3:

The same developer creates a very detailed and formally specified model of a software system using UML modelling language to explain how a complex system works.

Discuss whether this model can be considered a good conceptual model. Justify your reasoning with respect to the properties of conceptual modelling.

Answer 3:

A very detailed and formal model can be a good conceptual model, but only if it is fit for its purpose. Conceptual models rely on abstraction and reduction to manage and reduce complexity, so too much detail can reduce understandability and communication. At the same time, a conceptual model can extend reality if it helps in reasoning, analysis and validation. In such a case, a higher degree of detail and formalization is good. If this model is presented during early stages of a project and it fails to explain its intended stakeholders, it fails as a conceptual model regardless of its formal correctness. However, if the purpose of the model is

to support precise analysis, validation, or automated checks during the final stages of a project, a higher degree of formalization can be justified, as it improves reliability.

Question 4:

A developer observes an existing software system and creates a conceptual model to describe it. Later, another team member uses this model to understand how the system works, but comes to a different interpretation than the developer intended. Explain which effect has occurred during the creation and interpretation of the model.

Answer 4:

During the creation of the model, several transformation effects may occur. First, a transformation effect can arise during perception, because the developer observes the system selectively and may not notice all relevant aspects of reality. Next, a transformation effect occurs during conception, when the developer forms a mental model of the system and applies abstraction, selection, aggregation, and generalization. As a result, some details may be simplified or omitted. Finally, a transformation effect can occur during representation, because the modelling language may not be able to express every aspect of the developer's conception precisely, leading to loss or distortion of meaning.

During the interpretation of the model, further transformation effects can occur during interpretation, when the team member assigns meaning to the model's symbols based on their own understanding of the modelling language. If this understanding differs from that of the model creator, the intended meaning would differ. In addition, a transformation effect during association can occur when the interpreter links the interpreted concepts to the real-world system in a different or incorrect way, resulting in misunderstandings about how the system actually works.

Together, these transformation effects explain why different people can arrive at different interpretations of the same conceptual model, even if it is syntactically correct.

Question 5:

A simple modelling language is intended to describe a task management system. Use the following concepts to create your own modelling language:

- 1. A Task has a "title" and a "status".**
- 2. A User can be responsible for multiple Tasks.**
- 3. Each Task is assigned to exactly one User.**

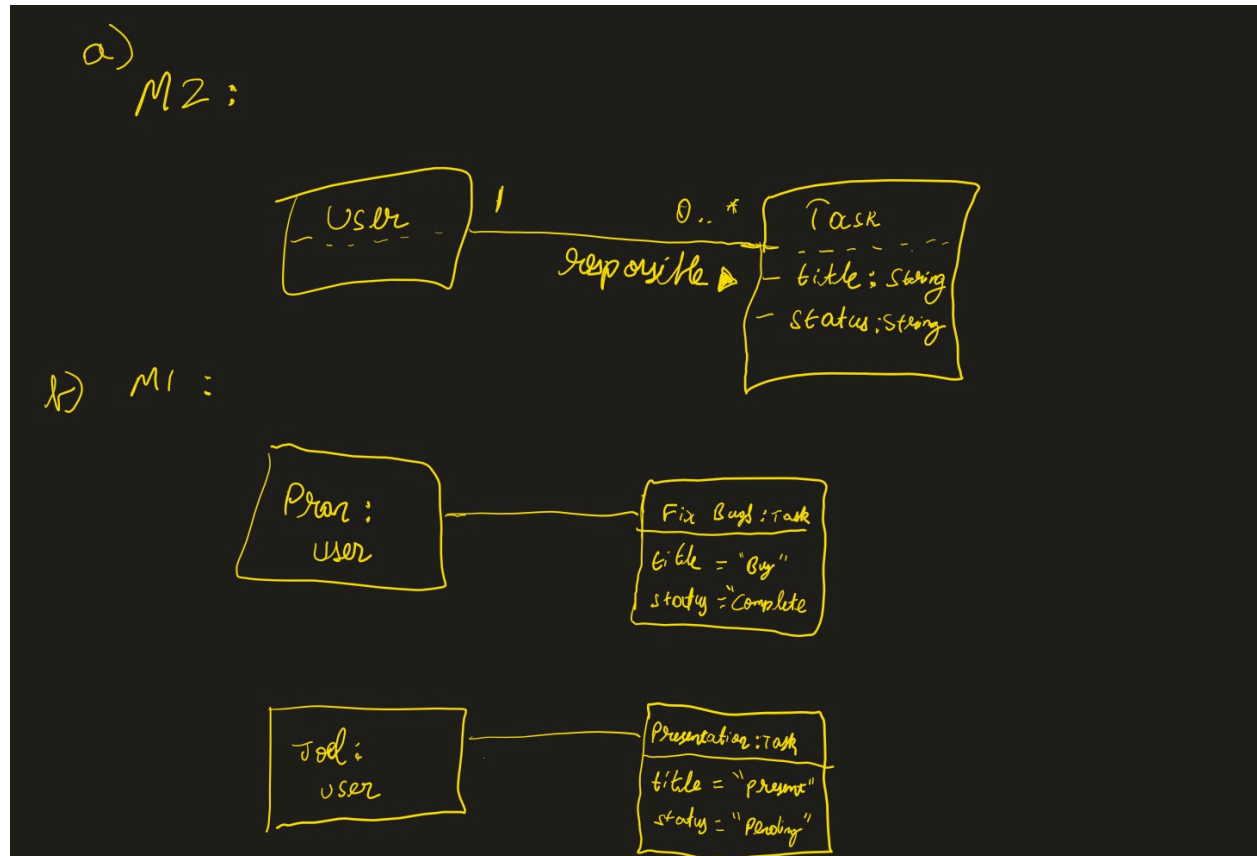
a) Draw a meta-model(M2) that defines the modelling language described above.

b) Using your meta-model, draw a small application model (M1) that contains 2 Users and 2 Tasks.

c) Briefly explain the relationship between the meta-model and the application model.

d) Identify the corresponding M3 (MOF) concepts for the main elements in your meta-model.

Answer 5:



c) The meta-model defines the modelling language, and the application model is an instance of that language.

d) User and Task are instances of MOF: Classifier

Title, status are instances of MOF: Property

User to task relationship instance of MOF: Association

Structure

1) ER Diagram: Car Rental Service

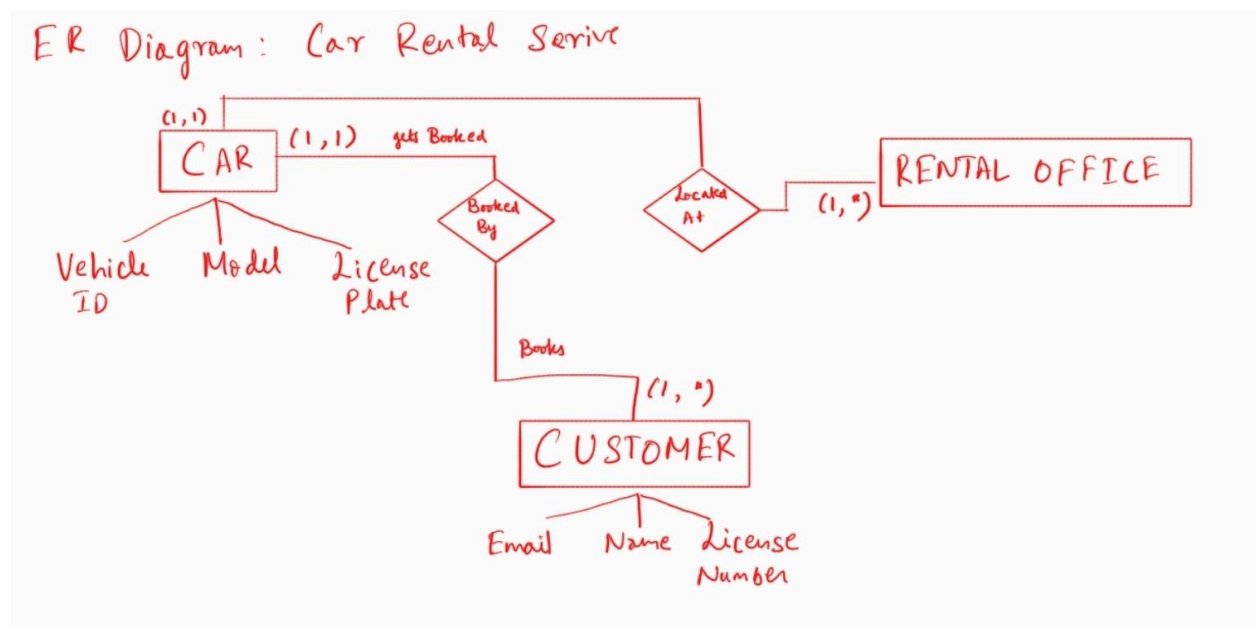
Task: Design an ER diagram for a "Car Rental Service" database.

- Requirements:
 - Entity Types: Define entities for CAR, CUSTOMER, and RENTAL_OFFICE.
 - Attributes: CAR should have a VehicleID (identifier), Model, and LicensePlate. CUSTOMER should have Email, Name, and LicenseNumber.
 - Relationship Types:

BOOKED_BY: Relates a CAR to a CUSTOMER. A customer can book multiple cars over time, but a car can only be booked by one customer at any given moment.

LOCATED_AT: Relates a CAR to a RENTAL_OFFICE. Each car must belong to exactly one office, while an office can house many cars.

- Cardinality Constraints: Specify the (min, max) constraints for each relationship to define whether participation is optional or mandatory.



2) An online learning platform (learning management system) supports two main kinds of users: **Students** and **Instructors**. Both share a common ID, name, and email. In addition, each Student records their registration date, while each Instructor is linked to one department. A **Course** is identified by a code and has a title and credit value. Every Course is taught by exactly one Instructor, though one Instructor may teach several Courses. Each Course consists of one or more **Modules**, and every Module contains several **Lessons**. Students can enroll in multiple Courses, and each Course can have many Students. For each enrollment the system must store the enrollment date and — at the end of the course — the student's final grade. Modules contain **Assignments**. A Student may submit multiple times for the same Assignment; each submission records the file location and the submission timestamp.

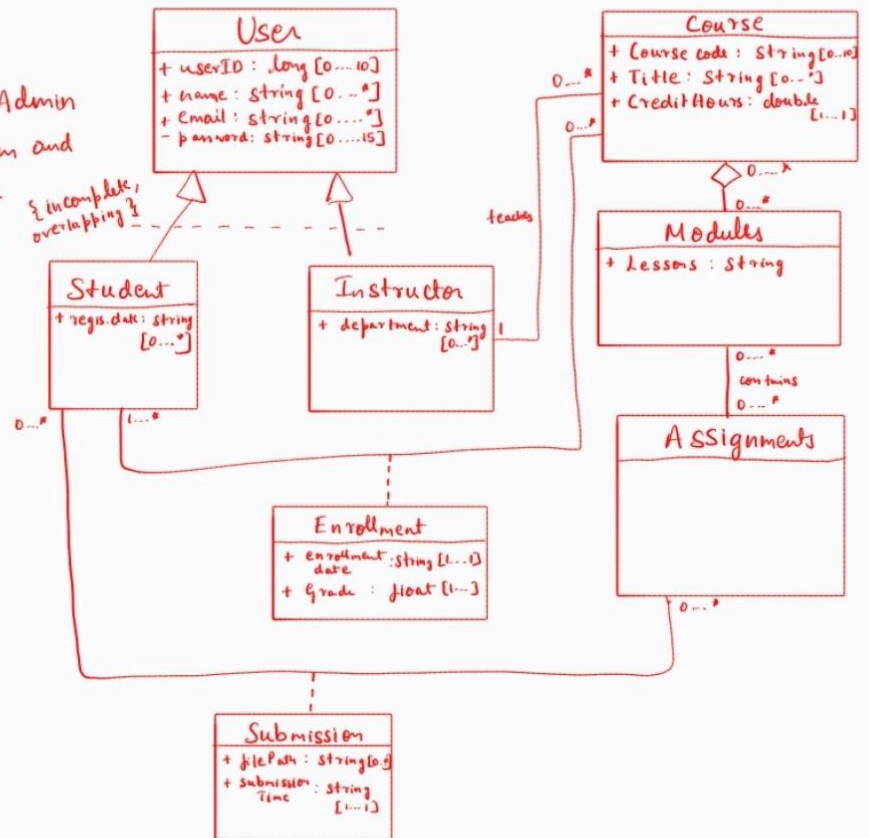
Task

Draw a **UML class diagram** showing the main classes, their key attributes, and all important relationships (including multiplicities, roles where helpful, generalization, composition, and any necessary association classes).

Make reasonable assumptions if something is not explicitly stated, but justify important design decisions (e.g. why you chose composition vs. aggregation, or why an association class was needed) briefly in notes on the diagram.

Learning management System

Incomplete because Admin can access this system and Overlapping because a PhD student can also be an instructor



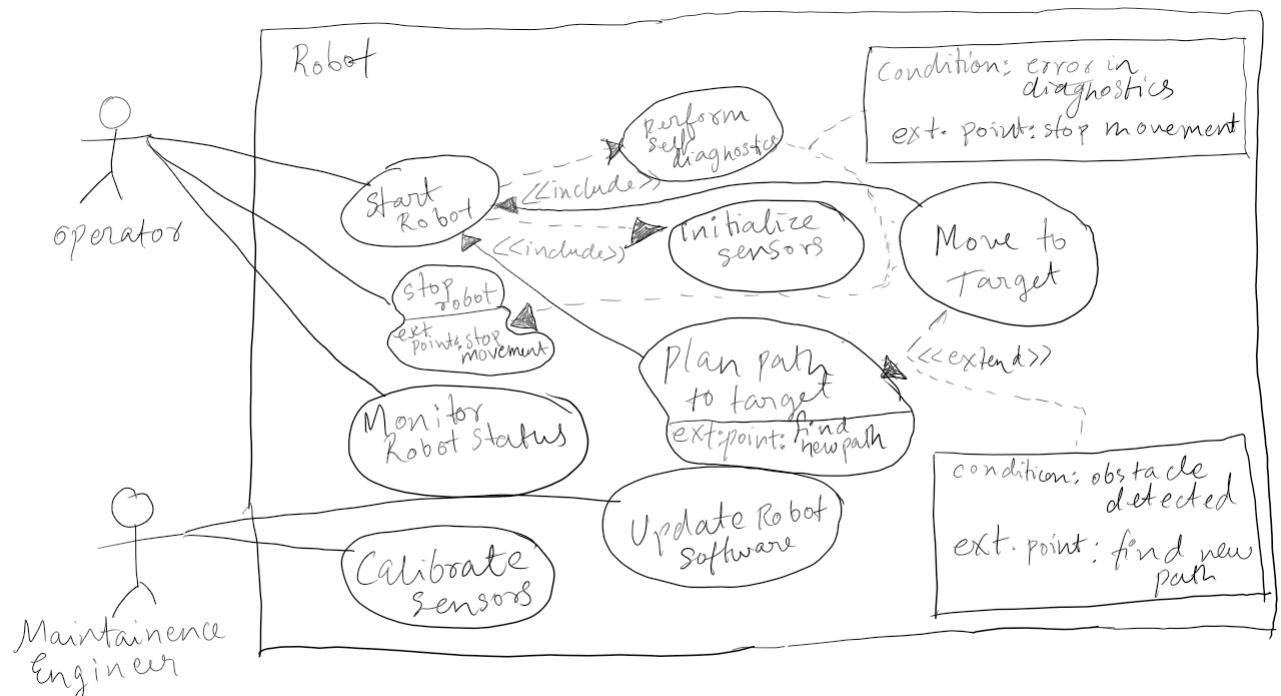
Function

Q.1) An Autonomous Mobile Robot (AMR) operates in a warehouse environment with the following requirements:

1. An **Operator** can:
 - Start the robot
 - Stop the robot
 - Monitor robot status
2. When the robot is started, it must:
 - Perform system self-diagnostics
 - Initialize sensors
3. During operation, the robot:
 - Navigates to a target location
 - Avoids obstacles
4. If an **obstacle is detected**, the robot must **replan its path**.
5. A **Maintenance Engineer** can:
 - Calibrate sensors
 - Update robot software

Create a Use Case Diagram to model the robot and it's usage.

A.1)



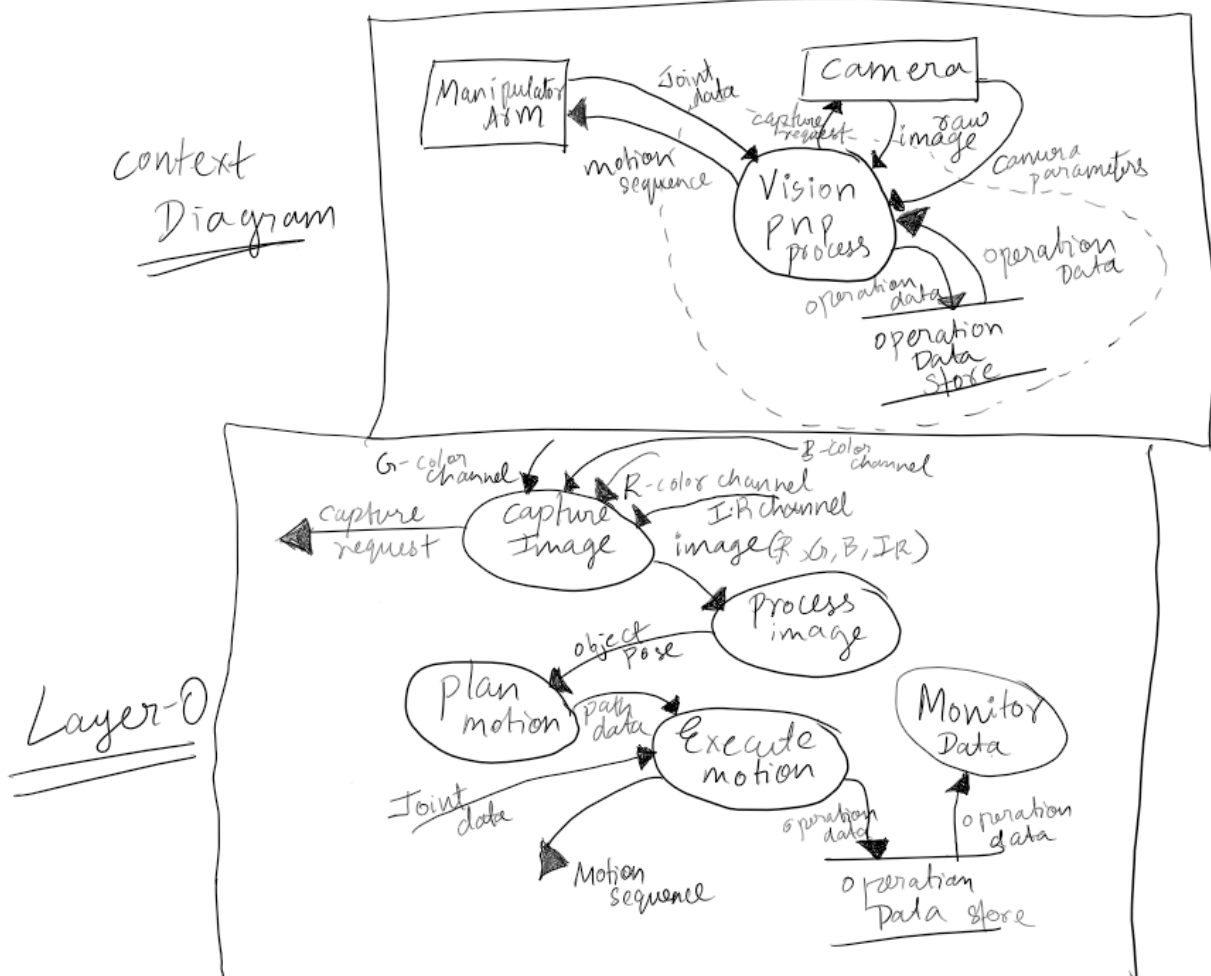
Q.2) A Vision-Based Pick-and-Place Robot System is used in a factory to identify parts on a conveyor belt and place them at predefined target locations.

The system operates as follows:

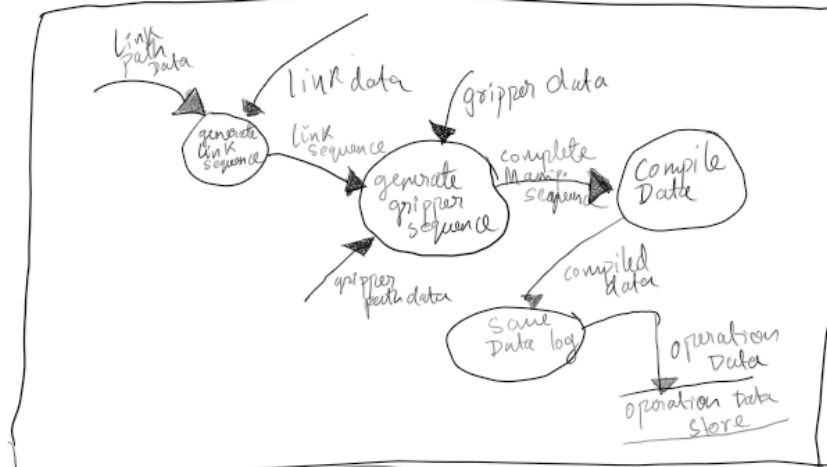
1. A **Camera** captures images of parts on the conveyor belt.
2. The system **processes the image** to **detect objects** and **extract their positions**.
3. Based on the detected object position, the system **plans a robot motion path**.
4. The robot **executes the motion** and **places the object** at the target location.
5. The system **logs operation data** for monitoring and analysis.

Model the mentioned system using a data flow diagram with at least two levels of abstraction, a data dictionary and give a mini specification for the primitive process of gripping an object.

A.2)

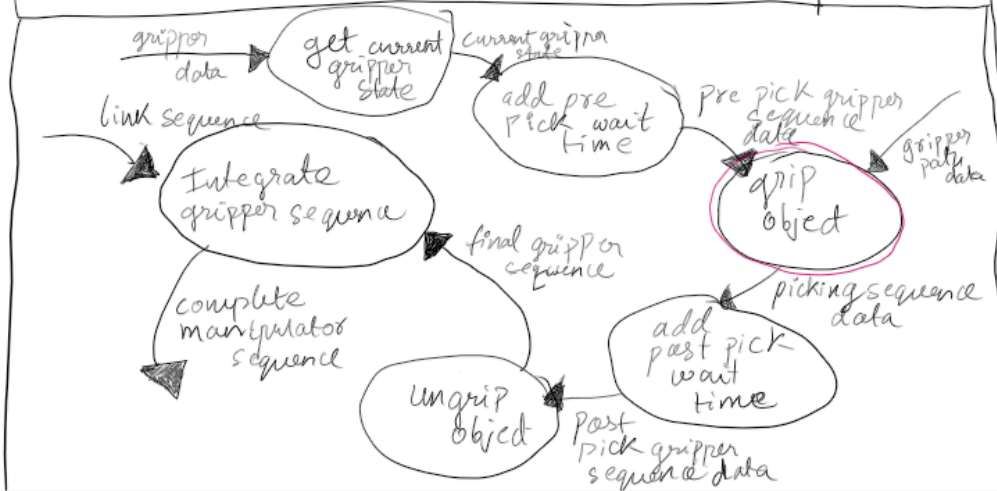


Layer 1:1
(Generate motion)

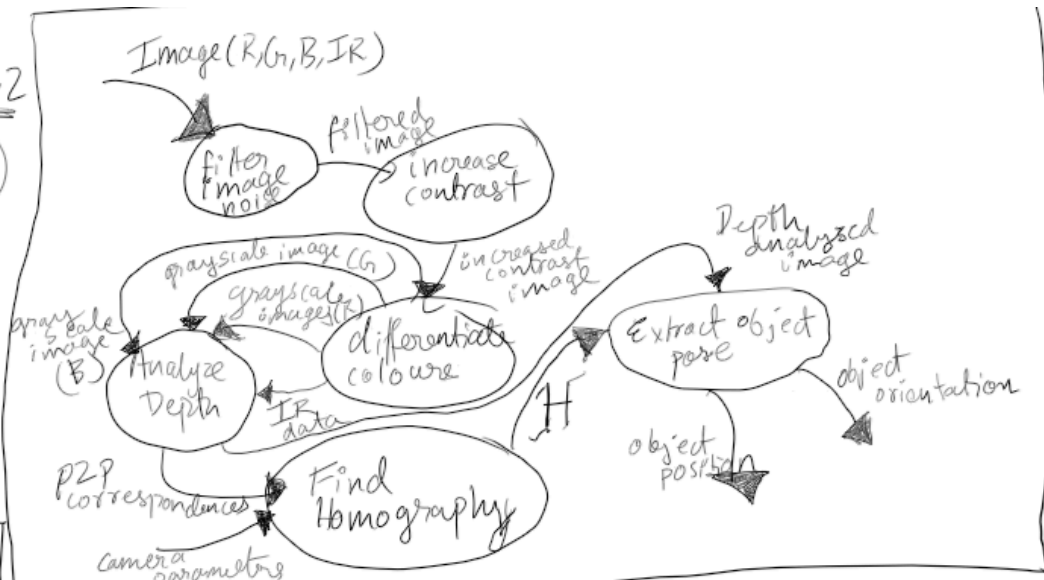


Layer 1:2
(process image)

Layer 2
(Generate gripper sequence)



Layer 1:2
(process image)



Data Dictionary

raw image = R-color channel
+ G-color channel
+ B-color channel
+ IR color channel

joint data = link data
+ gripper data

path data = link path data
+ gripper path data

object pose = {object position + ^{object} orientation}
* No transfer of new position /
orientation if NO CHANGE
in them from previous run *

object position = object center + object approach

object approach = object center + "20 cm" ^{point}

operation data = Raw image
+ image(R, G, B, IR)
+ path data
+ object pose
+ joint data

Mini specification for grip object

Once the pre pick gripper sequence data is ready, in order to grip object, the following must happen:

1. If the gripper is closed, first open it to it's maximum length of 20 cm.
2. Read the gripper path data to find out distance between object approach point and object center.
3. Add the delay corresponding to the calculated distance.
4. Then close the gripper at speed 1 cm/s until touch sensor on gripper confirms contact.
5. Halt teeth movement after contact and the object is now considered gripped.
6. Add the same delay as earlier so that the arm can move to the object approach point.

The grip object function has now completed the picking sequence.

Architecture

Q1) What is an architecture pattern? List some commonly used Architecture patterns.

Ans) An architecture pattern is a commonly recurring architectural designs. These patterns are documented and used to design new architectures. It gives a high level blueprint on what components are and how they interact with each other(layers, pipes, services etc)

Commonly used Architecture patterns include:

- Layered Architecture or Multi-Tier Architecture
- Model View Controller
- Pipe and Filter

Q2) What are the key rules of Layered Architecture? What are the benefits of using this architecture pattern? When is this pattern not suitable to use?

Ans)

The key rules for layered Architecture are:

- Every layer must be unique. There should be no overlapping between layers.
- Layers should only communicate with the layers above or below them.
- Layers should be individually maintainable.
- Each layer must be as independent as possible

Benefits of using Layered Architecture include:

- Reusability: It can be reused for any other application development as well
- Every layer is independently maintainable
- The data is very secure as it cannot be accessed directly in the user interface
- Each layer can be tested independently.
- High scalability.

Layered Architecture is not suitable to use when we want to develop:

- Real time performing systems.
- For small, simple systems. It becomes an overkill.
- Performance prioritised system due the architecture's low speed.

Q3) Describe the components/layers of the Model View Controller. State two advantages and two disadvantages of using this Architecture pattern.

Ans)

The Model View Controller consists of three components:

- Model: This layer contains the data and business logic. It provides methods to access and modify data.
- View: This layer shows the data to the User. This layer acts as the interface on which users can interact with the software.
- Controller: The Controller acts as a bridge between the Model and View. It takes inputs from the user interface and applies it on the model.

Two advantages of using this model are:

- Parallel development of layers is possible
- Highly Reusable for any interface based software.

Two disadvantages of using this are:

- Becomes complex for small size applications.

- Beginners could find it difficult to understand the working of MVC.

Q4) Classify each of the following as *multi-tier*, *pipe-and-filter*, *MVC*, or *plug-in* architecture, and justify briefly:

- a) A web application with a presentation layer, business logic layer, and data access layer that only call adjacent layers.**
- b) A compiler composed of a sequence of stages (lexing → parsing → semantic analysis → optimization → code generation).**
- c) A desktop application where a core framework loads external modules at runtime to extend functionality.**
- d) A GUI framework that separates domain logic, view, and input handling.**

Ans)a- Layered Architecture because the system is divided into stacked tiers and each tier is unique. The interaction only between neighboring/adjacent layers is a characteristic of Layered Architecture.

B- Pipe and Filter architecture because the process is divided into a linear sequence of independent filters connected by pipes. Each filter transforms a data stream to a new data stream which is taken to the next filter via a pipe for further processing.

D- This is Model View Controller architecture because the GUI distinguishes the model(domain), view (the visual representation) and the input handling being the controller.

Cyber Security

Question 1:

Security Analysis:

A University Online Learning System (E-learning) is used by students and teachers to:

- **Log in to the portal to view course content and exam registration.**
- **Upload and download assignments.**
- **Modify and Store exam result**
- **Conduct online exams**

The system runs on a central server and is accessed through the internet.

Name and briefly explain 3 security threats to this system. For each threat, propose one suitable security measure. Briefly describe how the university should systematically analyze and improve the security of this system.

Answer 1:

Security Threats:

- **Unauthorized access :** Attackers or students may log in using stolen passwords and access grades or exam papers.
- **Data leak:** Sensitive data such as student records and exam results could be stolen or exposed.
- **Reducing availability :** The system could be DDOS attacked , making it unavailable during exams or during submissions deadlines.

Security Measures:

- **Access control :** Use strong authentication such as complex passwords and two-factor authentication to ensure only valid users can log in.
- **Encryption :** Encrypt stored and transmitted data so that even if it is intercepted or stolen, it cannot be read.
- **Firewall protection :** Use firewalls, backups, and load-balancing to protect against attacks and ensure the system stays online.

Systematic Security Analysis:

First, identify important assets such as student data, exam results and unpublished exam questions etc... . Next, identify possible threats like hacking, data theft and system failure. Then assess the risks based on how likely and how damaging they could be. After that, choose suitable security controls such as access control, encryption, and backups. Finally, continuously monitor the system, update software, and review logs to detect and prevent new attacks.

Question 2:

Explain major drawbacks of using the Caesar Cipher for sending encrypted messages and suggest one way to improve its security.

Answer 2:

The Caesar Cipher is easy to break with some brute force methods as it is just a shift of characters by some length which is limited due to the small number of possible shifts before it starts repeating again (25 for English alphabets). It preserves letter frequency and length patterns. Common letters and words from a language would still appear as often as the original text in the encrypted text, making it easy to crack using frequency analysis.

Improvement suggestion:

To make better use of this method, it can be replaced with more complex ciphers, such as using different shift lengths for different lines of the text or even using different shifts for each letter. Decryption would still be possible with a secret key.

Quality Assurance

Q.1) Explain and distinguish between LOC, NLOC, NCSS. Explain why they are all poor software measures and more specifically poor measures of productivity

A.1)

Lines of Code

- Used for estimating development effort
- Lines of Code (LOC)
 - Disadvantage: Depending on the amount of comments
- Non-commenting Lines of Code (NLOC)
 - Disadvantage: Depending on the programming style
- Non Commenting Source Statements (NCSS)
 - Only source code specific characters are counted
E.g., ';' and '{' in Java Source Files

Lines of Code

- Advantages LOC/NCSS:
 - Very simple to calculate
 - LOC calculation is native to any UNIX
 - Command: wc -l
 - Special tools exist for NCSS
 - <http://www.kclee.de/clemens/java/javancss/>
- Disadvantages:
 - LOC/NCSS is dependent on the programming language
 - i.e. comparing LOC(python) with LOC(Java) is like comparing apples and pears [oranges]
 - Only size matters
 - Use of LOC for estimating productivity, effort, complexity is problematic

- For solving many things, poorly written code or under optimized code is often more lines of code. It is hence totally illogical to accredit someone with productivity for writing more lines of code compared to someone who solved the same task in much lesser lines.

- Can also talk about the pesticide paradoxon here as well where doing more is not the same as doing better/faster/more productively.
- People who write a lot of comments will also end up having more lines of code. However, writing well commented code is one of the best and most minimum programming practices to adhere to. So, if someone has less output but clearer and more maintainable output than one who doesn't add comments, discrediting them by counting NLOC is not sensible either.
- For the reasons above, any LOC is a poor measure of productivity.

Q.3) A software company develops embedded control software for industrial robots. Recently, the company has experienced an increase in post-deployment faults, leading to higher maintenance costs and customer dissatisfaction.

Management wants to understand and improve the quality of the software design, especially with respect to maintainability, in order to reduce future faults and maintenance effort.

Define the necessary Goals using the GQM goal definition template and give at least 3 Questions and use at least 3 Metrics to help answer them:

A.3) Goal 1

Analyse the	Software Product's Design
With Purpose of	Better Understanding
With Respect to	Maintainability and it's problems
In the View Point of	Management
In Context of	Embedded Control Software

Q.1. How complex is the software design?

M.1.1 Cyclomatic Complexity Number

Q.2. How modular and well-structured is the design?

M.2.1 Number of interfaces vs concrete classes

M.2.2 Average number of dependencies per class

Q.3. How much effort is required to modify or fix the software?

M.3.1 Average Developer Working hours

M.3.2 Average Time span of project development

Goal 2

Analyse the	Software Product's Design
With Purpose of	Better Improving
With Respect to	Maintainability and Fault Reduction
In the View Point of	Management
In Context of	Embedded Control Software

Q.1. Which parts of the software design are most difficult to maintain?

M.1.1 Average Cyclomatic Complexity Number per module

M.1.2 Percentage of modules exceeding a complexity threshold

Q.2. How does the current design structure contribute to fault occurrence?

M.2.1 Fault density per module (Faults per 1000 lines)

M.2.2 Number of post-deployment faults per release

Q.3. Does improving maintainability reduce maintenance effort over time?

M.3.1 Average Developer Working hours

M.3.2 Mean time to fix post-deployment faults

Q.3) Differentiate between Quality Management, Quality Planning, Quality Control, Quality Improvement and Quality Assurance.

A.3) Quality management is the coordination of Quality planning, control, improvement and assurance in order to ensure that quality with regard to the development of a system adheres to the specified quality requirements and standards.

Aspects of Quality Management

- Quality planning
- Quality control
- Quality improvement
- Quality assurance

[DIN EN ISO 9000:2015-11]

Quality Planning

Defines

- Quality goals
- Necessary execution processes
- Related resources
- Provable quality requirements
- Actions for quality control and quality assurance
- Quality management plans

Quality Control

- Goal: Ensure the quality
- Fulfilling the quality requirements
- Executing constructive actions
 - Prevent defects
- Executing analytical actions
 - Finding defects
- Constructive and analytical actions complement each other

Quality Improvement

- Evaluation of current quality assurance actions
 - Measuring
- Modification of quality planning
 - Goals, quality requirements, plans
- Modification of quality control
 - Actions, processes

Quality Assurance

Checking

- Fulfillment of quality requirements
- Effectiveness of the quality management systems (possibly certifications)
- Quality related measurements
- Processes and quality procedures